۸-۳ مشخصه‌ی بن‌بست

ما در بخش قبل نشان دادیم که چگونه بن‌بست می‌تواند در برنامه‌نویسی چندنخی با استفاده از قفل‌های انحصار متقابل اتفاق بیفتد. اکنون از نزدیک به شرایطی می‌پردازیم که بن‌بست را مشخص می‌کنند.

۸-۳-۱ شرایط لازم

بن‌بست در شرایطی به وجود می‌آید که همزمان، چهار شرط زیر در یک سیستم برقرار باشند:

۱- **انحصار متقابل:** بایستی حداقل منبعی باشد که در مد غیرقابل اشتراک گرفته می‌شود؛ یعنی، در هر لحظه تنها یک نخ می‌تواند از آن منبع استفاده کند. اگر نخ دیگری آن منبع را درخواست نماید، نخ درخواست‌کننده بایستی به تأخیر بیفتد تا این که منبع رها شود.

۲- **گرفتن و منتظر ماندن:** بایستی هر نخ حداقل یک منبع در اختیار داشته و به انتظار کسب منابع بیشتری باشد که فعلاً در اختیار نخ‌های دیگر هستند.

۳- **نبود پس‌گیری:** منابع نمی‌توانند پس‌گرفته شوند؛ یعنی، یک منبع می‌تواند تنها توسط نخی که آن را در اختیار دارد به صورت داوطلبانه بعد از این که نخ آن کار را تکمیل کرد، رها شود.

۴- **انتظار چرخشی:** بایستی یک مجموعه‌ای از نخ‌های منتظر $\{T_0, T_1, ..., T_n\}$ باشد که در آن، T_0 منتظر منبعی است که در اختیار T_1 قرار دارد، T_1 منتظر منبعی است که توسط T_2 اخذ شده، ...، T_{n-1} به انتظار منبعی است که توسط T_n گرفته شده و T_n منتظر یک منبع می‌باشد که در اختیار T_0 قرار دارد.

تأکید می‌کنیم که باید هر چهار شرط، همزمان برقرار شوند تا بن‌بست اتفاق بیفتد. شرط انتظار چرخشی، شرط گرفتن و منتظرماندن را لازم دارد، لذا هر چهار شرط از یکدیگر استقلال کامل ندارند. با این وجود، در بخش ۸-۵ خواهیم دید که بهتر است هر شرط را جداگانه بررسی کنیم.

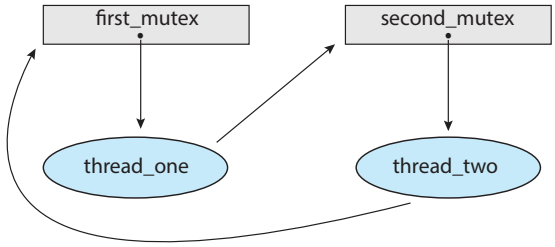
۸-۳-۲ گراف تخصیص منبع

می‌توان بن‌بست‌ها را به صورت دقیق‌تر با تصویر یک گراف جهت‌دار به نام **گراف تخصیص منبع سیستم** توصیف کرد. این گراف شامل مجموعه‌ی رئوس V و مجموعه‌ی یال‌های E می‌باشد. مجموعه‌ی رئوس V خود به دو مجموعه گره تقسیم می‌شود که عبارتند از: $T=\{T_1, T_2, ..., T_n\}$، شامل مجموعه‌ی تمامی نخ‌های فعال سیستم و $R=\{R_1, R_2, ..., R_m\}$، شامل مجموعه‌ی تمامی انواع منابع سیستم.

یک یال جهت‌دار از نخ T_i به منبع نوع R_j، به صورت $T_i \rightarrow R_j$ نشان داده می‌شود و بیانگر آن است که T_i یک نمونه از منبع نوع R_j را درخواست کرده و فعلاً منتظر آن منبع می‌باشد. یک یال جهت‌دار از منبع نوع R_j به نخ T_i، به صورت $R_j \rightarrow T_i$ نشان داده می‌شود و بیانگر آن است که یک

نمونه از منبع نوع R_j به نخ T_i تخصیص یافته است. یال جهت‌دار $P_i \rightarrow R_j$، **یال درخواست** و $R_j \rightarrow P_i$، **یال تخصیص**[1] نامیده می‌شوند.

در نمایش تصویری، هر یک از نخ‌های T_i را با یک دایره و هر منبع نوع R_j را با یک مستطیل نشان می‌دهیم. به‌عنوان یک مثال، گراف تخصیص منبع نشان داده شده در شکل ۸-۳، وضعیت بن‌بست برنامه‌ی شکل ۸-۱ را نمایش می‌دهد. چون امکان دارد که منبع نوع R_j بیش از یک نمونه داشته باشد، به ازای هر نمونه، داخل مستطیل R_j یک نقطه می‌گذاریم. توجه داشته باشید که یال درخواست، تنها به مستطیل R_j اشاره می‌کند، حال آنکه یال تخصیص بایستی از یکی از نقاط داخل مستطیل ترسیم شود.



شکل ۸-۳ گراف تخصیص منبع برنامه‌ی شکل ۸-۱.

موقعی که نخ T_i، یک نمونه از منبع نوع R_j را درخواست می‌کند، یک یال درخواست به گراف تخصیص منبع اضافه می‌شود. زمانی که این درخواست اعطا می‌شود، یال درخواست به یال تخصیص *نظیر* تبدیل می‌شود. زمانی که نخ، دیگر نیازی به دسترسی منبع ندارد، منبع را رها می‌سازد. در نتیجه، یال تخصیص حذف می‌شود.

گراف تخصیص منبع نشان داده شده در شکل ۸-۴، شرایط زیر را به تصویر می‌کشد.

- مجموعه‌های T، R و E:

$$T = \{T_1, T_2, T_3\}$$

$$R = \{R_1, R_2, R_3, R_4\}$$

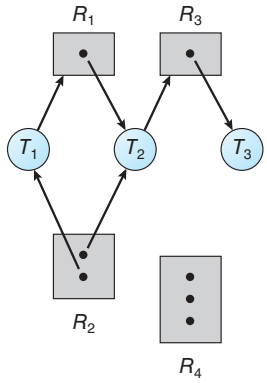
$$E = \{T_1 \rightarrow R_1, T_2 \rightarrow R_3, R_1 \rightarrow T_2, R_2 \rightarrow T_2, R_2 \rightarrow T_1, R_3 \rightarrow T_3\}$$

- نمونه‌های منابع:
  - یک نمونه از منبع نوع R_1
  - دو نمونه از منبع نوع R_2
  - یک نمونه از منبع نوع R_3
  - سه نمونه از منبع نوع R_4
- حالت‌های نخ‌ها:

1. Assignment edge

- نخ T_1 یک نمونه از منبع نوع R_2 را در اختیار دارد و منتظر نمونه‌ای از منبع نوع R_1 می‌باشد.
- نخ T_2 یک نمونه از منبع نوع R_1 و یک نمونه از R_2 در اختیار دارد و منتظر نمونه‌ای از منبع نوع R_3 می‌باشد.
- نخ T_3 یک نمونه از R_3 را در اختیار دارد.

می‌توان با داشتن تعریف گراف تخصیص منبع نشان داد که اگر گراف فاقد دور باشد، آنگاه هیچ یک از نخ‌های سیستم در بن‌بست قرار ندارند. اگر گراف دارای دور باشد، آنگاه ممکن است بن‌بست وجود داشته باشد.



شکل ۸-۴ گراف تخصیص منبع.

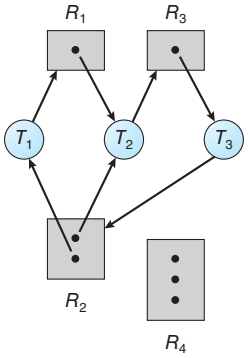
اگر هر نوع منبع دقیقاً یک نمونه داشته باشد، در این صورت وجود یک دور نشان می‌دهد که یک بن‌بست اتفاق افتاده است. اگر دور فقط شامل مجموعه‌ای از انواع منابعی باشد که هر یک دارای یک نمونه هستند، در این صورت بن‌بست رخ داده است. هر نخی که در این دور باشد در بن‌بست قرار می‌گیرد. در این حالت، وجود دور در گراف، شرط لازم و کافی برای وقوع بن‌بست است.

اگر هر نوع منبع چندین نمونه داشته باشد، در این صورت وجود دور الزاماً به معنی وقوع بن‌بست نیست. در این حالت، وجود یک دور در گراف شرط لازم برای وقوع بن‌بست است اما کافی نیست.

برای روشن‌شدن این مفهوم، دوباره به تصویر گراف تخصیص منبع شکل ۸-۴ بر می‌گردیم. فرض کنید که نخ T_3، یک نمونه از منبع نوع R_2 را درخواست می‌کند. چون، فعلاً نمونه‌ای از منبع موجود نیست، یال درخواست $T_3 \rightarrow R_2$ را به گراف اضافه می‌کنیم (شکل ۸-۵). در اینجا، دو دور مینیمال در سیستم به وجود می‌آید:

$$T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$$
$$T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2$$

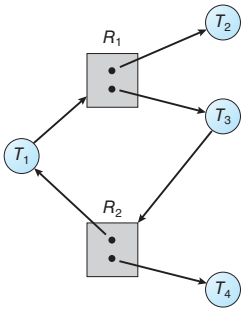
نخ‌های T_1، T_2 و T_3 در بن‌بست قرار دارند. نخ T_2 منتظر منبعی است که T_3 در اختیار دارد. نخ T_3 نیز منتظر رهایی منبع R_2، توسط یکی از نخ‌های T_1 و یا T_2 می‌باشد. به علاوه، نخ T_1 منتظر نخ T_2 می‌باشد تا منبع R_1 را رها کند.



شکل ۸-۵ گراف تخصیص منبع با یک بن‌بست.

حال، گراف تخصیص منبع شکل ۸-۶ را در نظر بگیرید. در این مثال، یک دور مشاهده می‌کنیم:

$$T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$$



شکل ۸-۶ گراف تخصیص منبع با یک دور، اما فاقد بن‌بست.

با این وجود، هیچ بن‌بستی در کار نیست. مشاهده می‌شود که نخ T_4 می‌تواند نمونه‌ی منبع R_2 را که در اختیار دارد رها کند. سپس آن منبع می‌تواند به T_3 تخصیص یابد که با این کار، دور شکسته می‌شود.

به طور خلاصه، اگر یک گراف تخصیص منبع فاقد دور باشد، در این صورت سیستم در حالت بن‌بست قرار ندارد. اگر دوری وجود داشته باشد، در این صورت امکان دارد سیستم در حالت بن‌بست قرار بگیرد یا در حالت بن‌بست نباشد. این مشاهده در مواجهه و برخورد با مسأله‌ی بن‌بست حائز اهمیت است.

اکنون، گراف تخصیص منبع شکل ۷-۳ را در نظر بگیرید. در این مثال نیز دور زیر را داریم:

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

۸-۴ روش‌های رسیدگی به بن‌بست‌ها

به بیان کلی، با یکی از سه روش زیر می‌توانیم با مسأله‌ی بن‌بست برخورد کنیم:

- می‌توانیم به طور کلی از مسأله چشم‌پوشی کرده و وانمود کنیم که سیستم هرگز با بن‌بست مواجه نمی‌شود.
- می‌توانیم برای پیشگیری یا اجتناب از بن‌بست از یک پروتکل استفاده کنیم که اطمینان می‌دهد سیستم هرگز در حالت بن‌بست واقع نمی‌شود.
- می‌توانیم اجازه بدهیم که سیستم وارد حالت بن‌بست شود، سپس آن را آشکار کرده و رفع کنیم.

راه‌حل اول به عنوان یکی از روش‌هایی است که در بیشتر سیستم‌عامل‌ها، از جمله لینوکس و ویندوز استفاده می‌شود. پس، بر عهده‌ی برنامه‌نویسان کاربردی و هسته است که برنامه‌هایی برای رسیدگی به بن‌بست‌ها نوعاً با استفاده رویکردهای مطرح شده در راه‌حل دوم بنویسند. برخی سیستم‌ها—همچون پایگاه‌داده‌ها—راه‌حل سوم را انتخاب می‌کنند که به بن‌بست‌ها اجازه می‌دهد تا اتفاق بیفتند و سپس، ترمیم را انجام می‌دهند.

در ادامه، هر یک از این سه روش رسیدگی به بن‌بست‌ها را به اختصار بحث می‌کنیم. سپس، الگوریتم‌های تفصیلی را در بخش‌های ۸-۵ تا ۸-۸ مطرح می‌کنیم. اما، پیش از هر چیز لازم به ذکر است که برخی محققان بر این باورند که هیچ یک از این روش‌های پایه نمی‌توانند به تنهایی برای تمامی طیف مسائل تخصیص منبع سیستم‌عامل‌ها مناسب باشند. با این همه، ترکیب روش‌های مبنایی، این اجازه را به ما می‌دهد که در هر یک از رده‌های منابع موجود سیستم، روش بهینه‌ای را انتخاب کنیم.

سیستم می‌تواند برای تضمین این که، هرگز بن‌بستی اتفاق نخواهد افتاد، طرح پیشگیری و یا اجتناب از بن‌بست را در پیش بگیرد. **پیشگیری از بن‌بست**[1]، مجموعه‌ای از روش‌ها برای تضمین این است که حداقل یکی از شرایط لازم (بخش ۸-۳-۱) نمی‌تواند برقرار شود. این روش‌ها با اعمال محدودیت بر روی چگونگی انجام درخواست‌ها با بن‌بست‌ها مقابله می‌کنند. این روش‌ها را در بخش ۸-۵ بحث می‌کنیم.

اجتناب از بن‌بست[2] لازم دارد که پیشاپیش، اطلاعات بیشتری در رابطه با منابعی که نخ‌ها در طول زمان اجرای خودشان درخواست و استفاده خواهند نمود، در اختیار سیستم‌عامل گذاشته شود. سیستم‌عامل با داشتن این معلومات می‌تواند در رابطه با هر یک از درخواست‌ها، در مورد اعطای منبع و یا منتظرماندن نخ تصمیم‌گیری کند. سیستم با نظر به منابع موجود فعلی، منابع در اختیار هر یک از نخ‌ها، و درخواست‌ها و رهاکردن‌های آتی منابع هر یک از نخ‌ها، در رابطه با اعطای درخواست و یا به تأخیرانداختن آن تصمیم‌گیری می‌کند. این طرح‌ها را در بخش ۸-۶ بحث می‌کنیم.

اگر سیستمی از هیچ یک از روش‌های پیشگیری و یا اجتناب از بن‌بست استفاده نکند، در این صورت امکان رخداد شرایط بن‌بست وجود دارد. در چنین محیطی، سیستم می‌تواند با استفاده از

1. Deadlock prevention
2. Deadlock avoidance

یک الگوریتم، حالت سیستم را برای تعیین رخداد بن‌بست بررسی نموده و از الگوریتم دیگری برای رفع بن‌بست استفاده نماید (اگر به واقع بن‌بستی اتفاق افتاده باشد). این ملاحظات را در بخش ۸-۷ و بخش ۸-۸ بحث می‌کنیم.

با نداشتن الگوریتم‌هایی برای تشخیص و ترمیم از بن‌بست ممکن است به وضعیتی برسیم که سیستم در یک حالت بن‌بست قرار گرفته و حتی راهی برای تشخیص این که چه چیزی اتفاق افتاده است نیز در پیش رو نداریم. در این حالت، بن‌بست تشخیص داده نشده باعث تنزل کارایی سیستم می‌شود، چون منابع سیستم در اختیار نخ‌هایی است که نمی‌توانند اجرا شوند و نخ‌های بیشتری نیز باز هم با درخواست آن منابع وارد حالت بن‌بست می‌شوند. بالاخره، سیستم از کار می‌افتد و بایستی دوباره به صورت دستی راه‌اندازی شود.

اگرچه به نظر می‌رسد مشکل مسأله‌ی بن‌بست با این روش حل نمی‌شود، اما همان‌گونه که اشاره شد در بیشتر سیستم‌عامل‌ها استفاده دارد. هزینه به عنوان یک دغدغه‌ی مهم است. چشم‌پوشی از احتمال وقوع بن‌بست‌ها از رویکردهای دیگر ارزانتر است. چون در بسیاری از سیستم‌ها، بن‌بست‌ها خیلی کم پیش می‌آیند (بر فرض، یکبار در ماه)، هزینه‌ی اضافی روش‌های دیگر ممکن است به صرفه نباشد.

به علاوه، روش‌هایی که برای ترمیم از سایر شرایط عاقبت به‌خیری، همچون قفل زنده، استفاده می‌شوند ممکن است برای ترمیم از بن‌بست هم استفاده شوند. در برخی شرایط، یک سیستم از شکست عاقبت به‌خیری رنج می‌برد، اما یک حالت بن‌بست نیست. برای مثال، این وضعیت را می‌توان ناشی از یک نخ بلادرنگ دید که با بالاترین الویت اجرا می‌شود (یا هر نخی که با یک زمانبند بدون پس‌گیر اجرا دارد) و هرگز کنترل را به سیستم‌عامل بر نمی‌گرداند. در چنین شرایطی بایستی سیستم روش‌های ترمیم دستی داشته باشد و حتی می‌تواند از این تکنیک‌ها به سادگی برای رفع بن‌بست نیز استفاده کند.

۸-۵ پیشگیری از بن‌بست

همان‌گونه که در بخش ۸-۳-۱ متذکر شدیم، برقراری هر چهار شرط برای وقوع بن‌بست ضرورت دارد. اگر به این اطمینان برسیم که حداقل یکی از این شرط‌ها نمی‌تواند برقرار شود می‌توانیم از بن‌بست به دور باشیم. در اینجا، مطابق این رویکرد، هر یک از این چهار شرط لازم را مورد بررسی قرار می‌دهیم.

۸-۵-۱ انحصار متقابل

شرط انحصار متقابل بایستی برقرار شود. یعنی، حداقل یک منبع می‌بایست اشتراک‌ناپذیر باشد. منابع اشتراک‌پذیر، نیازی به دسترسی انحصاری نداشته و لذا نمی‌توانند در یک بن‌بست واقع شوند. فایل‌های فقط‌خواندنی مثال خوبی برای منابع اشتراک‌پذیر به حساب می‌آیند. اگر چندین نخ بخواهند یک فایل فقط‌خواندنی را همزمان باز کنند، دسترسی همزمان فایل قابل اعطا می‌باشد. یک نخ هرگز نیازی به انتظار منبع اشتراک‌پذیر نمی‌بیند. با این وجود، در حالت کلی نمی‌توان با نقض شرط انحصار متقابل از بن‌بست‌ها پیشگیری کرد، چون برخی از منابع ذاتاً